

# AgentKube

## Kubernetes for AI Agents: A 16-Week Build Plan

An open-source agent runtime platform. You build the infrastructure that AI agents run on: execution engine, tool gateway, memory service, Kafka-backed communication, observability, policy enforcement, RAG, and production deployment. Every AI agent concept learned through building production-grade distributed infrastructure.

## Table of Contents

1. [Profile & Constraints](#)
2. [What Is AgentKube](#)
3. [System Architecture Overview](#)
4. [Repository Structure](#)
5. [Tech Stack \(Full Detail\)](#)
6. [Why Not LangChain](#)
7. [Services Running Locally](#)
8. [Topic Coverage Map](#)
9. [Phase 1: Agent Execution Engine](#)
10. [Phase 2: Tool Gateway Service](#)
11. [Phase 3: Scheduler & Planning Service](#)
12. [Phase 4: Memory Service](#)
13. [Phase 5: Kafka-Backed Agent Communication](#)
14. [Phase 6: Observability Stack](#)
15. [Phase 7: Policy Engine & Auth](#)
16. [Phase 8: Resilience & Chaos Layer](#)
17. [Phase 9: Knowledge Service \(RAG\)](#)
18. [Phase 10: Deploy & Demonstrate](#)
19. [Free Tier Strategy](#)
20. [Common Mistakes to Avoid](#)
21. [Real-World Use Cases](#)
22. [Learning Resources by Phase](#)
23. [Architecture Doc Outline \(Interview Prep\)](#)
24. [Quick Reference One-Line Phase Summaries](#)

## Profile & Constraints

| Field                  | Detail                                                           |
|------------------------|------------------------------------------------------------------|
| <b>Goal</b>            | Switch into AI/ML engineering or platform engineering roles      |
| <b>Current Role</b>    | Backend engineer (microservices, distributed systems)            |
| <b>Primary Stack</b>   | Java (Spring Boot, gRPC, Kafka), Python, Rust (basics)           |
| <b>Time Budget</b>     | 10-15 hrs/week over 16 weeks (~168-248 total hours)              |
| <b>LLM Budget</b>      | Gemini Flash free tier, Ollama local, Claude free tier           |
| <b>End Deliverable</b> | Deployed platform, documented GitHub repo + Interview-ready demo |
| <b>LLM Experience</b>  | Dabbled, built a chatbot, used LLM APIs a few times              |

## What Is AgentKube

AgentKube does for AI agents what Kubernetes does for containers.

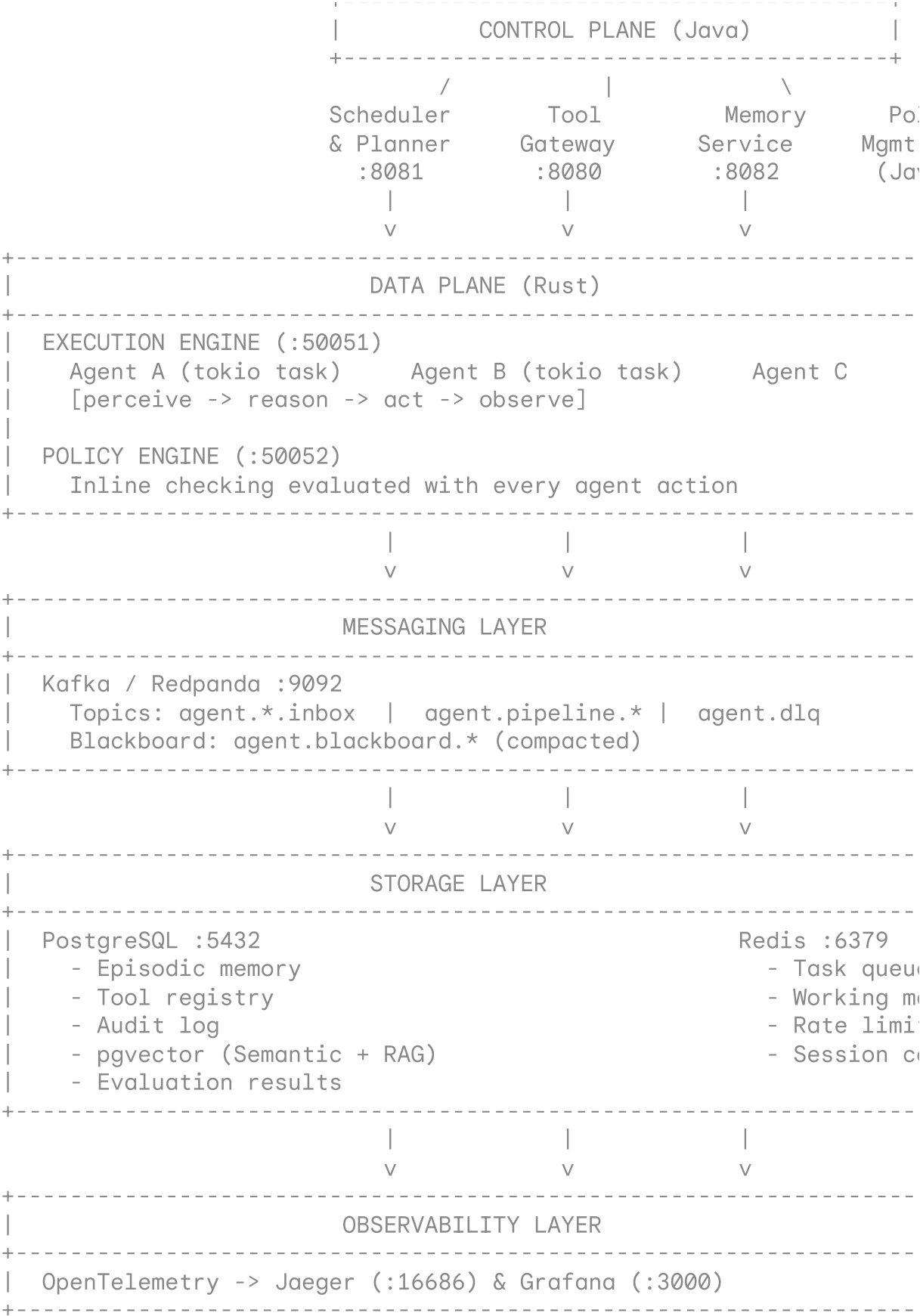
When people build AI agents today, they hardcode everything: the agent logic, the tools it calls, how it talks to other agents, and where it stores memory. Every project reinvents the same plumbing. AgentKube is that plumbing, built properly as a set of microservices.

You write an agent definition in YAML, submit it to the platform, and the platform handles:

- **Running it:** The execution engine manages the perceive-reason-act loop.
- **Giving it tools:** The tool gateway handles discovery, validation, and rate limiting.
- **Giving it memory:** The memory service provides working, episodic, and semantic memory.
- **Letting it talk to other agents:** Kafka-backed communication with pub/sub, fan-out, and fan-in functionality.
- **Watching it:** Distributed tracing, cost tracking, and quality evaluation.
- **Constraining it:** A dedicated policy engine enforces what each agent can and cannot do.
- **Giving it knowledge:** RAG service provides searchable knowledge-base access.
- **Keeping it alive:** Circuit breakers, chaos testing, and graceful degradation protocols.

The real reason to build this: when you interview for an AI platform engineering role, you don't say *"I built a chatbot with LangChain."* You say *"I built the infrastructure layer that agents run on."* That's a completely different conversation.

## System Architecture Overview



How a Request Flows Through the System

- 1. A multi-step mission payload is sent to the Control Plane.

an LLM-generated DAG JSON script.

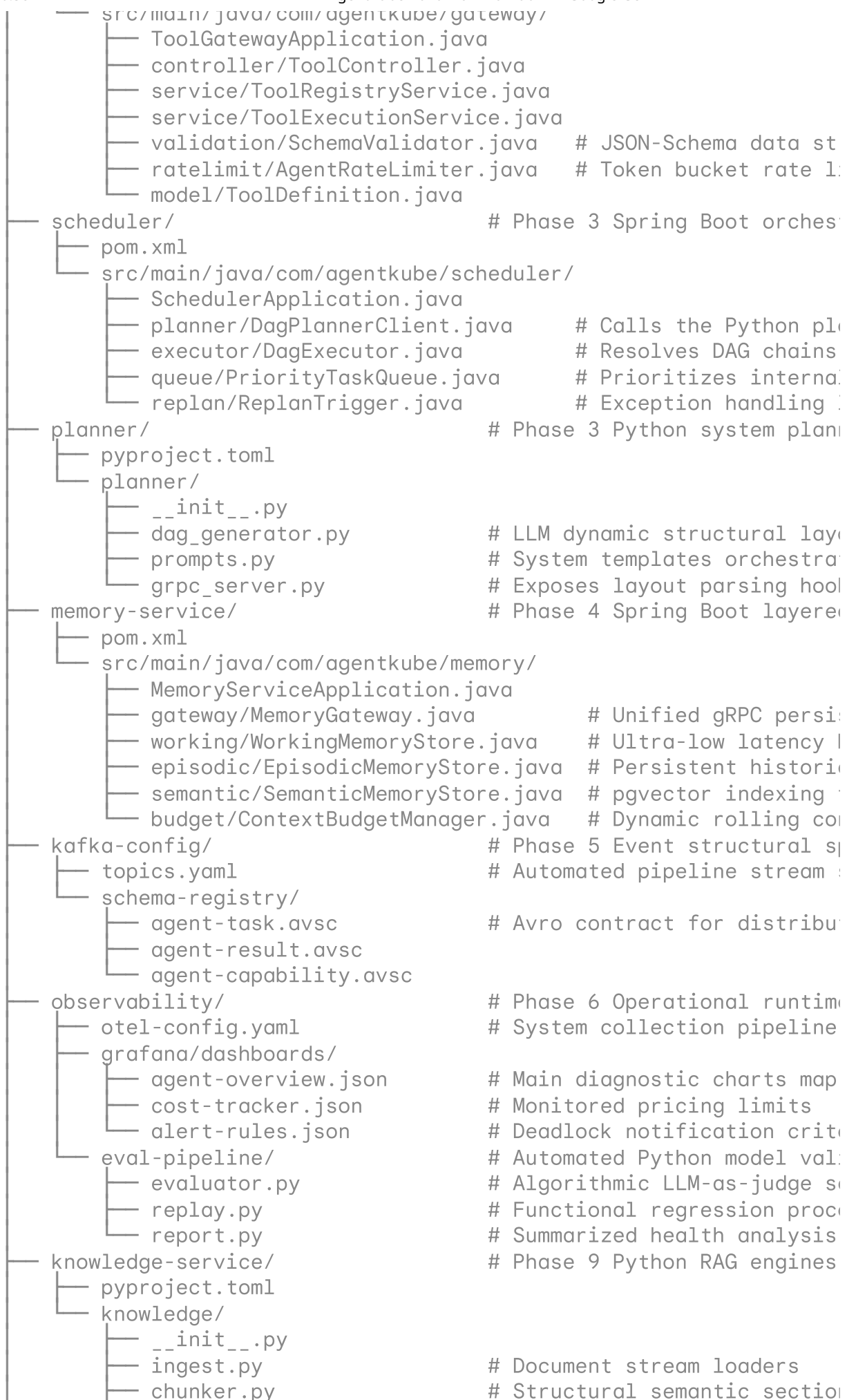
3. The Scheduler coordinates tasks, walking the DAG execution tree and dispatching payloads to the engine.
4. The Agent Execution Engine loads the YAML configuration, instantiates a dedicated `tokio` isolation thread task, and enters its core perceive-reason-act lifecycle loop.
5. **Tool Calls:** Agent reasons *"I need to search"* and issues a gRPC request to the **Tool Gateway (:8080)**. The gateway handles schema validation, authentication, rate-limiting, runs the tool execution proxy, and returns results safely.
6. **Memory Sync:** Agent reasons *"Let me check what I know"* and reaches out to the **Memory Service (:8082)**. The service performs a vector recall bounded by token budgets and returns relevancy-ranked blocks.
7. **Policy Enforcement:** Every outbound step or tool call hits the inline **Policy Engine (:50052)**. It evaluates constraints (e.g., spending bounds or tool access permissions) with sub-millisecond overhead ( $< 0.2\text{ms}$ ).
8. **Inter-agent Messaging:** Agent results route through **Kafka/Redpanda (:9092)** pipeline routing keys, pushing data directly into downstream downstream inboxes.
9. **Observability:** Distributed OpenTelemetry traces stream out to **Jaeger (:16686)** and metrics flow into **Grafana (:3000)** for live tracking of LLM costs, system health, and execution loops.
10. **Evaluation:** Upon task completion, an offline Python Evaluation Pipeline invokes an LLM-as-a-judge scoring method, checking accuracy and saving findings back to Postgres.

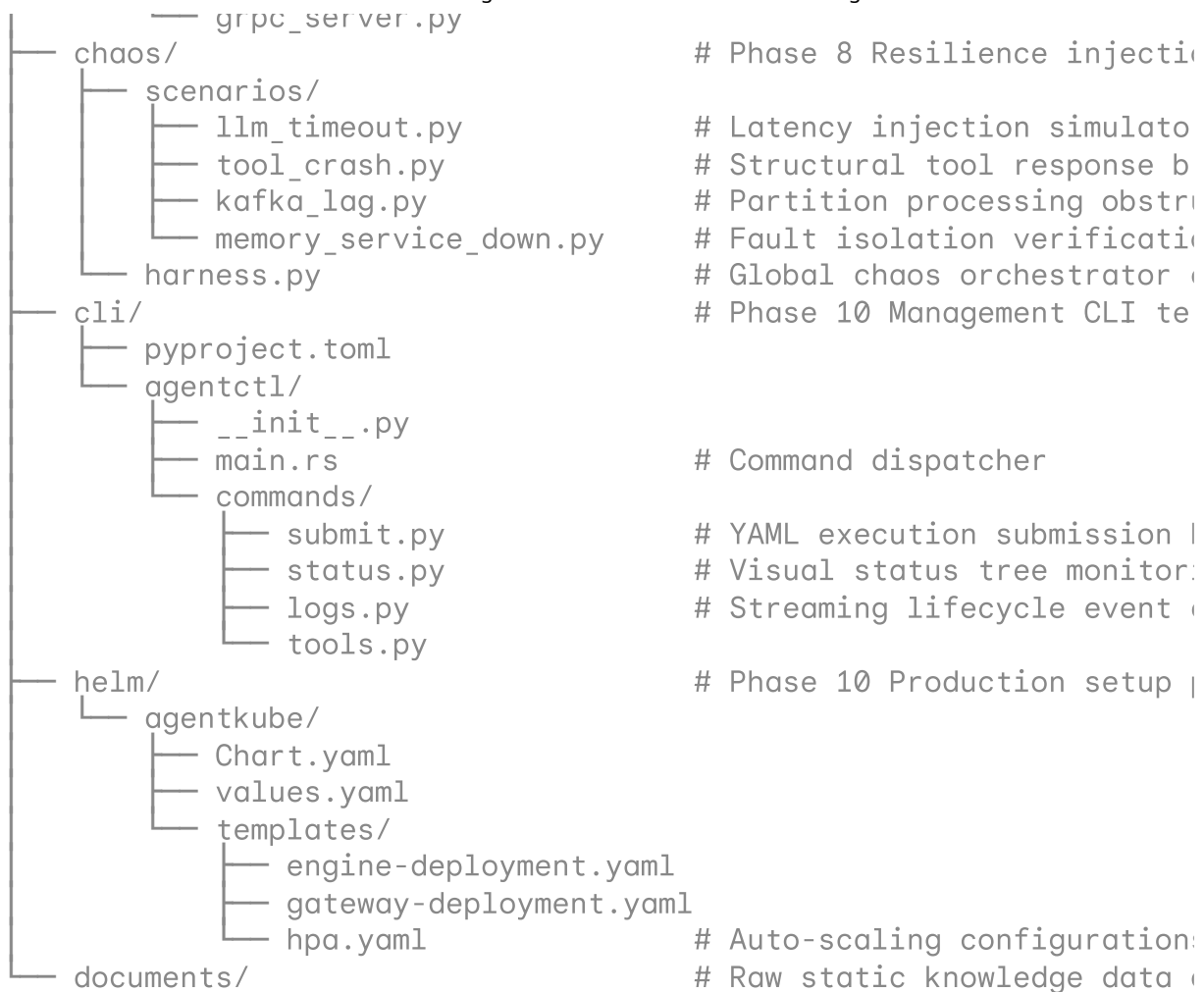
## Repository Structure

```

agentkube/
├── README.md                # Main project interview summary
├── docker-compose.yml       # One command to run all playground
├── architecture.md          # In-depth system design blueprint
├── proto/                   # Shared strict language-agnostic
│   ├── agent.proto          # gRPC endpoints for lifecycle
│   ├── tool.proto           # Schema contracts for structured
│   ├── memory.proto         # Unified memory management
│   ├── planner.proto        # DAG orchestration hooks
│   └── knowledge.proto      # Context retrieval schemas
├── engine/                  # Phase 1 Rust agent execution
│   ├── Cargo.toml
│   └── src/
│       ├── main.rs          # Orchestrates runtime startup
│       ├── agent_process.rs # Isolation management primitives
│       ├── execution_loop.rs # ReAct state machine loop (L
│       ├── llm_client.rs    # High-performance token/stream
│       ├── config.rs        # Declarative configuration
│       └── policy_engine.rs # Phase 7 fast inline policy
└── tool-gateway/           # Phase 2 Spring Boot tool service

```





## Tech Stack (Full Detail)

### Data Plane (Performance-Critical, Inline)

- **Rust:** Used for the core Execution Engine and Policy Engine. High concurrency with ultra-low latency, strict compile-time safety for state transitions, minimal memory overhead ( $< 20\text{MB}$  idle base memory footprint).
- **Libraries:** tokio (async runtime), tonic (gRPC implementation), serde / serde\_yaml (declarative config parser), request (async HTTP connection pooling for streaming LLM outputs).

### Control Plane (Business Logic, Orchestration)

- **Java 21 & Spring Boot 3.x:** Manages system coordination, metadata registry tracking, scheduling engines, and data pipeline layers. Excellent type safety and mature client libraries for Kafka and enterprise integration patterns.
- **Libraries:** Spring Cloud Gateway (for tool routing layers), Resilience4j (circuit breakers and adaptive rate-limit structures), io.grpc:grpc-netty-shaded.

- **Python 3.11+:** Leveraged for data manipulation, dynamic task decomposition planning modules, RAG pipelines, and verification scripts.
- **Libraries:** `litellm` (standardized provider interface), `pydantic` (structural payload checking models), `fastapi` / `grpcio` (high-throughput application serving points), `langsmith` / `ragas` (system evaluation reporting components).

Storage & Messaging

- **PostgreSQL 16:** Used as the central transaction record store, audit logging registry, and episodic ledger tracking point.
- **pgvector:** Extends Postgres to store embeddings, serving semantic memory queries and processing hybrid vector exploration steps.
- **Redis 7.2:** Powers ultra-fast active operational memory caches, real-time rate limit tracking counters, and ephemeral task coordination lists.
- **Apache Kafka / Redpanda:** Acts as the high-throughput, horizontally scaleable event spine for multi-agent asynchronous pub/sub data processing.

Why Not LangChain

AgentKube avoids heavy frameworks like LangChain by design. Here is why infrastructure abstraction levels matter:

| LangChain Abstractions | What AgentKube Builds          | Why It Matters                                                                                                                   |
|------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| AgentExecutor          | <b>Rust Execution Engine</b>   | You control the internal state machine, timeouts, and process isolation barriers directly.                                       |
| LangChain Tools        | <b>Tool Gateway Service</b>    | Adds JSON Schema contract verification, centralized access control, and API rate-limiting proxies.                               |
| LangChain Memory       | <b>Layered Memory Store</b>    | Implements multi-tier storage paths (Redis/Postgres), dynamic context optimization, and structural isolation boundaries.         |
| LangChain Chains       | <b>Kafka Message Pipelines</b> | Built as independent distributed systems with robust features like retries, backpressure handling, and Dead Letter Queues (DLQ). |

Services Running Locally

The local environment spins up 12 standard infrastructure containers with a single command:

```
docker-compose up -d
```

docker-compose up -d

| Service Name     | Language | Network Port | Development Phase | Operational Core Purpose                                               |
|------------------|----------|--------------|-------------------|------------------------------------------------------------------------|
| Engine           | Rust     | :50051       | Phase 1           | Direct lifecycle execution and ReAct orchestration loops.              |
| Tool Gateway     | Java     | :8080        | Phase 2           | Unified proxy for registration, validation, and execution controls.    |
| Scheduler        | Java     | :8081        | Phase 3           | Manages operational workflows and executes multi-agent DAG schedules.  |
| Memory Service   | Java     | :8082        | Phase 4           | Multi-tier gRPC memory interface layer (Working/Episodic/Semantic).    |
| Redpanda (Kafka) | C++      | :9092        | Phase 5           | High-speed message bus handling communication between running agents.  |
| Redpanda Console | Go       | :8080        | Phase 5           | Visual dashboard for monitoring topics, consumer lag, and payloads.    |
| Jaeger           | Go       | :16686       | Phase 6           | Distributed OpenTelemetry trace aggregation across services.           |
| Grafana          | Go       | :3000        | Phase 6           | System telemetry dashboards, error logging, and cost metrics.          |
| PostgreSQL       | C        | :5432        | All               | Central relational engine and pgvector embedding storage backend.      |
| Redis            | C        | :6379        | All               | Ephemeral task queues, lock management, and distributed rate limiting. |
| Policy Engine    | Rust     | :50052       | Phase 7           | Ultra-fast inline evaluation of rules and resource limits.             |



Service

semantic chunk retrieval  
vectors.

Topic Coverage Map

|                                   |                                   |
|-----------------------------------|-----------------------------------|
| +-----                            |                                   |
|                                   | AGENTKUBE 16-WEEK DEVELOPMENT MAP |
| +-----                            |                                   |
| Phase 1: Rust Execution Engine    | Phase 6: Observability (OTel      |
| Phase 2: JSON-Schema Tool Gateway | Phase 7: OPA Policy-as-Code       |
| Phase 3: DAG Planning/Scheduler   | Phase 8: Chaos Injection Lay      |
| Phase 4: Tiered Memory Architect  | Phase 9: Hybrid Retrieval R       |
| Phase 5: Kafka Multi-Agent Core   | Phase 10: Helm/K8s Production     |
| +-----                            |                                   |

1. **Agent Architectures (Phase 1):** ReAct execution loop designed as a deterministic Rust state machine, implementing an "agent-as-a-process" model via declarative YAML configurations.
2. **Tool Use & Function Calling (Phase 2):** Enforces strict JSON Schema tool contracts, enabling dynamic discovery, automated data validation, and rate-limited executor patterns.
3. **Planning & Decomposition (Phase 3):** Generates execution DAGs via LLM calls, handling parallel topological tasks, graph resolution, and replan-on-failure traps.
4. **Memory Systems (Phase 4):** 3-tier system logic isolating contexts via ultra-low latency Redis frames, linear Postgres logs, and deep pgvector databases.
5. **Multi-Agent Coordination (Phase 5):** Distributed event routing via dedicated Kafka inbox partitions, compacted blackboard topics, and Dead Letter Queues (DLQ).
6. **Evaluation & Monitoring (Phase 6):** OpenTelemetry tracking capturing granular sub-span latency records, parsing usage billing metrics into Grafana.
7. **Guardrails & Safety (Phase 7):** Fast inline evaluation maps validating safety assertions against inputs and outputs via high-speed code paths.
8. **Failure Modes & Debugging (Phase 8):** Hardened resilience architecture using loop-detection filters, fallback handlers, and simulated network partitions.
9. **Agentic RAG (Phase 9):** Advanced platform-native indexing incorporating hybrid query engines, reranking steps, and explicit source citation tracking.
10. **Productionization (Phase 10):** Native infrastructure scaling using Helm blueprints and Kubernetes targets, backed by active consumer lag metrics.

Phase 1: Agent Execution Engine

- **Topic:** Agent Architectures & Design Patterns

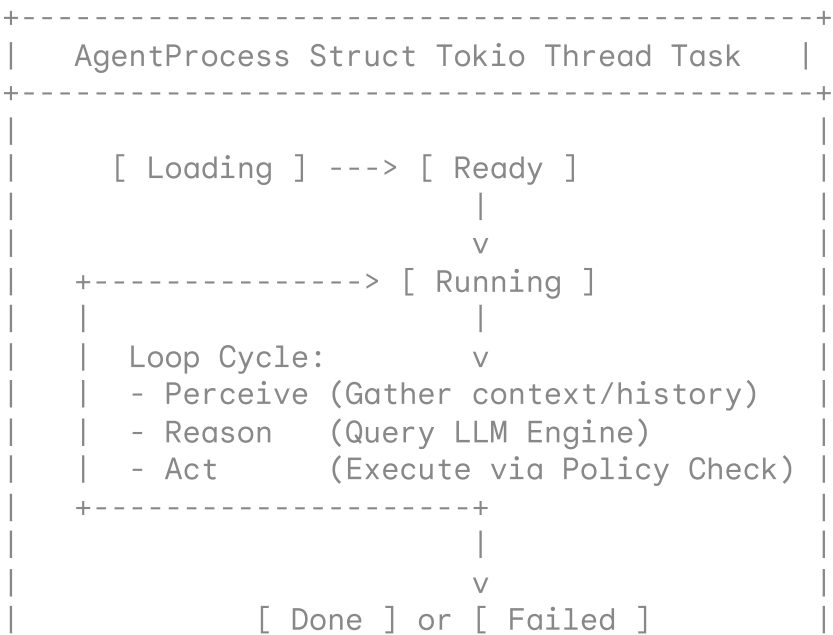
- **Timeline:** Weeks 1-2
- **Hours:** 20-30 hours
- **Prerequisites:** Basic Rust concepts (ownership model, enums, async/await futures), foundational gRPC concepts.
- **Real-world Analogy:** containerd manages the lifecycles of container processes; this engine manages the lifecycles of AI agent processes.

What It Is

The core kernel of the platform. A high-performance Rust binary that orchestrates agent lifecycles. It loads declarative agent configurations from YAML files, runs the perceive-reason-act loop as a deterministic state machine, and exposes a strict gRPC control API. Every upstream orchestrator or microservice interfaces with this engine to deploy and manage agents.

What You Build

- **gRPC Interface:** Exposes structural control APIs like `RunAgent` , `PauseAgent` , `KillAgent` , `GetAgentStatus` ,and `StreamLogs` .
- **AgentProcess Struct State Machine:** Explicit transitions: `Loading` → `Ready` → `Running` → `Paused` → `Done` / `Failed` . Invalid state changes are caught at compile-time as explicit type errors.
- **Execution Engine Loop:** Captures environment observations, formats prompt histories, requests streaming LLM structures, evaluates action components, and strictly enforces step/task timeouts.
- **Process Isolation:** Spawns each active agent worker inside an isolated `tokio` task thread bounded by dedicated context token tracking lengths and thread cancellation tokens.



+-----+

## Sample Agent Definition

```
# examples/agents/researcher.yaml
apiVersion: agentkube/v1
kind: Agent
metadata:
  name: researcher
  labels:
    team: content
spec:
  model: gemini-flash
  system_prompt: |
    You are a research agent. Given a topic, search for information
    read relevant sources, and produce structured findings.
    Always cite your sources.
  tools:
    - web_search
    - file_read
    - http_request
  resources:
    max_memory: 50MB
    max_tokens_per_task: 5000
    timeout_per_step: 30s
    timeout_per_task: 300s
  restart_policy: on_failure
  max_restarts: 3
```

## Core Rust State Specification

```
// engine/src/agent_process.rs
use std::sync::Arc;
use tokio::sync::watch;
use tokio::util::CancellationToken;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum AgentState {
    Loading,
    Ready,
    Running,
    Paused,
    Done,
    Failed,
}

pub struct AgentProcess {
```

```
pub struct AgentConfig {
```

```

pub config_path: String,
pub state_tx: watch::Sender<AgentState>,
pub state_rx: watch::Rx<AgentState>,
pub cancel_token: CancellationToken,
}

impl AgentProcess {
    pub fn new(id: String, config_path: String) -> Self {
        let (state_tx, state_rx) = watch::channel(AgentState::Loading);
        Self {
            id,
            config_path,
            state_tx,
            state_rx,
            cancel_token: CancellationToken::new(),
        }
    }

    pub async fn transition_to(&self, next: AgentState) -> Result<()> {
        let current = *self.state_rx.borrow();
        if Self::is_valid_transition(current, next) {
            let _ = self.state_tx.send(next);
            Ok(())
        } else {
            Err(format!("Invalid state transition from {:?} to {:?}", current, next))
        }
    }

    fn is_valid_transition(from: AgentState, to: AgentState) -> bool {
        match (from, to) {
            (AgentState::Loading, AgentState::Ready) => true,
            (AgentState::Ready, AgentState::Running) => true,
            (AgentState::Running, AgentState::Paused) | (AgentState::Paused, AgentState::Running) => true,
            _ => false,
        }
    }
}

```

## Sample gRPC Protobuf Contract

```

// proto/agent.proto
syntax = "proto3";
package agentkube.v1;

service AgentService {
    rpc RunAgent (RunAgentRequest) returns (RunAgentResponse);
    rpc PauseAgent (PauseAgentRequest) returns (PauseAgentResponse);
    rpc KillAgent (KillAgentRequest) returns (KillAgentResponse);
    rpc GetAgentStatus (GetAgentStatusRequest) returns (GetAgentStatusResponse);
}

```

```
}

message RunAgentRequest {
    string agent_id = 1;
    string yaml_definition = 2;
}

message RunAgentResponse {
    string job_id = 1;
    string status = 2;
}

message PauseAgentRequest { string agent_id = 1; }
message PauseAgentResponse { bool success = 1; }

message KillAgentRequest { string agent_id = 1; }
message KillAgentResponse { bool success = 1; }

message GetAgentStatusRequest { string agent_id = 1; }
message GetAgentStatusResponse {
    string agent_id = 1;
    string state = 2;
    int64 tokens_used = 3;
    int64 running_time_ms = 4;
}

message StreamLogsRequest { string agent_id = 1; }
message StreamLogsResponse {
    int64 timestamp = 1;
    string level = 2;
    string source_component = 3;
    string log_message = 4;
}
```

## Phase 2: Tool Gateway Service

- **Topic:** API Gateway Patterns & Schema Validation
- **Language:** Java (Spring Boot)
- **Timeline:** Weeks 3-4
- **Hours:** 15-20 hours

### What It Is

An abstraction layer for tools. Instead of letting agents execute raw system utilities directly, all tool invocations run as isolated proxy calls through this gateway. It exposes a clean API catalog, validates parameters against JSON Schema rules, maps execution identities, and tracks operational health.

- **Dynamic Schema Validator:** Parses custom JSON structural profiles at startup, checking inbound parameter shapes against contracts before firing backend worker calls.
- **Security & Auth Controls:** Implements token-bucket rate limits per agent class, shielding downstream external services from execution loops.
- **Unified Tool Registry Interface:** Standard components ( `web_search` , `calculator` , `db_query` ) expose JSON-Schema descriptors to let the Planner construct accurate execution calls.

## Phase 3: Scheduler & Planning Service

- **Topic:** Distributed Workflows & Directed Acyclic Graphs (DAGs)
- **Language:** Java (Orchestrator) + Python (Planner Engine)
- **Timeline:** Weeks 5-6
- **Hours:** 25-30 hours

### What It Is

The high-level brains of the cluster. When complex objectives hit the platform, the service decomposes the task into a dependency graph of sub-tasks. The Scheduler executes this graph, managing state changes and processing independent branches in parallel.

### What You Build

- **Python Planner Daemon:** Translates structural multi-line user intents into execution-ready JSON graph scripts.
- **Java Graph Orchestrator:** Validates, processes, and walks the topological map, dispatching work items into high-priority execution queues.
- **Dynamic Failure Re-planner:** Catches downstream processing faults, intercepting errors to trigger sub-graph rewrites and recovery paths.

## Phase 4: Memory Service

- **Topic:** Multi-Tier Storage Architectures & Context Window Management
- **Language:** Java (Spring Boot) + Redis + Postgres + pgvector
- **Timeline:** Week 7
- **Hours:** 15-20 hours

### What It Is

Provides stateful persistence for agents. It manages context windows by dividing data across three storage tiers, balancing retrieval speed, scale, and semantics.

### What You Build

conversation turns of a running agent instance.

- **Append-Only Episodic Ledger:** Linear Postgres database tracking complete transaction histories for compliance and debugging.
- **Semantic Vector Index:** Binds pgvector components to parse document chunks, updating context budgets via relevance scoring.

## Phase 5: Kafka-Backed Agent Communication

- **Topic:** Event-Driven Microservices & Message Brokers
- **Language:** Java + Apache Kafka / Redpanda
- **Timeline:** Week 8
- **Hours:** 15-20 hours

### What It Is

The messaging spine of the cluster. It moves multi-agent orchestration away from fragile synchronous HTTP calls, using an asynchronous event-driven message bus instead.

### What You Build

- **Isolated Topic Topologies:** Maps messaging layout rules ( `agent.*.inbox` , `agent.pipeline.*` ), configuring retention limits and partition counts.
- **Compacted Blackboard Store:** Implements high-performance, globally shared state spaces via Kafka log compaction configurations.
- **Fault Recovery Layers:** Implements robust error boundaries by defining explicit poison-pill payload traps and Dead Letter Queues (DLQ).

## Phase 6: Observability Stack

- **Topic:** Distributed Tracing, Structured Logging, and System Metrics
- **Language:** Java / Rust Core + OpenTelemetry + Grafana
- **Timeline:** Week 9
- **Hours:** 15-20 hours

### What It Is

Provides deep visibility into the system. It tracks spans across microservice boundaries, monitoring token consumption, costs, and response times.

### What You Build

- **OpenTelemetry Interceptors:** Injects distributed tracing contexts across systems, mapping dependencies from the orchestrator to the LLM.
- **Central Telemetry Dashboard:** Visualizes model metrics, tracing live token costs and alerting on pipeline failures or runaway agent loops.

pattern to score pipeline accuracy.

## Phase 7: Policy Engine & Auth

- **Topic:** Enterprise Guardrails, Role-Based Access Control (RBAC), and Security
- **Language:** Rust (Fast Evaluator) + Java (Management Plane)
- **Timeline:** Week 10
- **Hours:** 15-20 hours

### What It Is

The security core of AgentKube. It acts as an inline policy firewall, evaluating tool access permissions, budget compliance, and data leaks before actions execute.

### What You Build

- **Fast Inline Evaluator:** A lightweight Rust module built into the execution engine to enforce declarative security policies.
- **Token Budget Guard:** Monitors and blocks requests that exceed dollar spending thresholds or resource allocations.
- **Injection Defense Filters:** Sanitizes prompt inputs and outputs to prevent instruction overrides or data exfiltration.

## Phase 8: Resilience & Chaos Layer

- **Topic:** Fault Tolerance, Chaos Engineering, and Self-Healing
- **Language:** Java + Python Chaos Suite
- **Timeline:** Week 11
- **Hours:** 15-20 hours

### What It Is

The defensive layer of the platform. It uses stability patterns to protect the system against common failure modes like rate-limiting blocks, network drops, and unresponsive models.

### What You Build

- **Adaptive Circuit Breakers:** Wraps outbound API calls to isolate failing dependencies and trigger fallback processing.
- **Loop-Detection Filter:** Monitors and terminates agents caught in repetitive execution states or unproductive cycles.
- **Automated Chaos Suite:** Simulates infrastructure failures (e.g., database drops, API timeouts) to verify self-healing behavior.

## Phase 9: Knowledge Service (RAG)



- **Language:** Python + pgvector
- **Timeline:** Weeks 12-13
- **Hours:** 20-25 hours

## What It Is

A high-performance retrieval service that extends agent context. It provides access to large document sets using semantic chunking, indexing, and hybrid search.

## What You Build

- **Semantic Document Chunker:** Splits unstructured files along logical topic transitions rather than fixed character counts.
- **Hybrid Ensemble Search:** Combines dense vector retrieval (pgvector) with keyword matching (BM25) using Reciprocal Rank Fusion (RRF).
- **Citation Tracker:** Inspects retrieved segments to ensure answers explicitly reference valid source documents.

## Phase 10: Deploy & Demonstrate

- **Topic:** Cloud Infrastructure, DevOps, and Portfolio Presentation
- **Language:** Helm / Kubernetes Configuration scripts + Python CLI tool
- **Timeline:** Weeks 14-16
- **Hours:** 30-40 hours

## What It Is

The final production packaging phase. It converts the multi-container configuration into production-grade Helm charts ready for deployment to any Kubernetes engine. You compile a comprehensive CLI tool ( `agentctl` ) to easily manage agent lifecycles, and write a stellar flagship repository README detailing every architectural trade-off to impress technical interviewers.

## What You Build

- **Platform Management CLI:** A command-line tool ( `agentctl` ) to submit agent definitions, view status trees, and stream logs.
- **Helm Deployment Blueprint:** Packages configurations for Kubernetes, setting up horizontal pod autoscaling based on consumer lag.
- **End-to-End Demo Setup:** Pre-configures a multi-agent scenario (e.g., a collaborative research team) to demonstrate the platform for interviews.

## Free Tier Strategy

service mappings:

- **Inference Models:** Run small models locally via **Ollama** (e.g., `llama3.8b` or `phi3`) to develop loop mechanics offline without incurring costs. For advanced tasks, use **Gemini Flash APIs** via Google AI Studio's free tier.
- **Infrastructure Components:** Run your databases, message brokers, and telemetry collectors locally inside lightweight **Docker containers** instead of provisioning paid cloud infrastructure.
- **Testing & Infrastructure Frameworks:** Use local instances of **Jaeger** and **Grafana** for monitoring. Run your evaluation pipelines against open-source evaluation tools to score data profiles for free.

## Common Mistakes to Avoid

- **Relying on High-Level Frameworks:** Avoid building core scheduling or execution logic with LangChain or AutoGen wrappers. The goal of this project is to build the underlying platform mechanics from scratch.
- **Ignoring Context Window Limits:** Do not append conversation history indefinitely. Without dynamic context budget optimization, agents will quickly exhaust their token limits and fail.
- **Synchronous Microservice Design:** Avoid connecting services via cascading HTTP calls. Synchronous designs create tight coupling and risk cascading failures if a single service encounters latency.
- **Skipping Step-Level Timeouts:** Always enforce timeouts on every individual step. Without strict execution limits, a stuck API call can lock up resources and leave worker threads stranded indefinitely.

## Real-World Use Cases

- **Autonomous Technical Analysis:** Decomposes a software feature request into independent sub-tasks, queries codebase repositories, analyzes dependencies, and generates pull requests.
- **Market Trend Aggregation:** Coordinates specialized research agents to scrape data, clean financial inputs, run comparative statistical scripts, and compile comprehensive market reports.
- **Automated Customer Operations:** Parses inbound customer issues, validates account history via secure tools, applies behavioral safety rules, and generates responses while tracking overall operational costs.

## Learning Resources by Phase

- **Phases 1 & 7 (Rust Core):** Read *The Rust Programming Language* (specifically chapters on `async/await` futures and ownership models). Study the documentation for the `tonic` framework to learn gRPC development patterns.

Review the documentation for `Resilience4j` to understand circuit breakers, rate limiters, and fault-tolerance patterns.

- **Phases 5 & 8 (Event Streams & Chaos Engineering):** Read *Designing Data-Intensive Applications* by Martin Kleppmann (specifically chapters focusing on distributed messaging, partitioning, and system consensus).
- **Phases 6 & 9 (Observability & RAG):** Explore the official OpenTelemetry documentation to learn about distributed tracing. Review papers on *Reciprocal Rank Fusion (RRF)* and hybrid search architectures.

## Architecture Doc Outline (Interview Prep)

When presenting AgentKube to engineering interviewers, structure your system design documentation using this framework to highlight critical trade-offs:

1. **Executive Problem Statement:** Explain how standard agent frameworks create tight coupling between agent logic and core system plumbing, and why a microservice runtime platform scales better.
2. **Data Plane vs. Control Plane Isolation:** Detail the trade-offs of using high-performance Rust for inline data processing alongside Spring Boot for complex orchestration logic.
3. **Storage & Memory Layering Choices:** Explain your storage design choices—specifically why you separated volatile working memory in Redis from persistent episodic logs in PostgreSQL.
4. **Event Spine vs. Synchronous REST:** Contrast your event-driven Kafka architecture with traditional synchronous HTTP patterns, explaining how it helps handle backpressure and isolate component failures.

## Quick Reference One-Line Phase Summaries

- **Phase 1 (Engine):** Builds a high-performance Rust state machine to manage the core agent loop.
- **Phase 2 (Gateway):** Secures and validates tool invocations using an API proxy with strict JSON Schema enforcement.
- **Phase 3 (Scheduler):** Decomposes complex user goals into executable dependency DAGs.
- **Phase 4 (Memory):** Implements a tiered memory structure across Redis, PostgreSQL, and pgvector.
- **Phase 5 (Messaging):** Connects agents using an asynchronous, event-driven Kafka message bus.
- **Phase 6 (Telemetry):** Adds distributed OpenTelemetry tracing to track performance and model execution costs.
- **Phase 7 (Policy):** Enforces fine-grained compliance and safety rules directly inline.